

SI A BASE DE COMPOSANTS

Dr GOITA . Y

Servlets & Java Server Page (JSP)

- 1 Les servlets
 - Conteneur
 - Cycle de vie
 - L'API Servlet
- 2 Java Server Pages (JSP)
 - Module Web

SERVLET

- Programme Java s'exécutant côté serveur Web Pour des pages HTML dynamiques Exécution dans un espace isolé (Web Application) Spécification :
- Sun (sous partie de JavaEE) Implémentation de référence : Apache Group (Jakarta Tomcat)

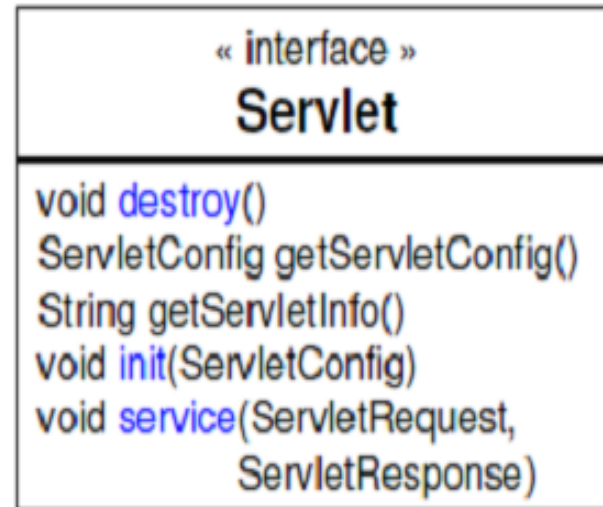
Conteneur

- Un conteneur est un composant logiciel système qui contrôle d'autres composants, dits métier
- Tomcat est un exemple de conteneur
- Les servlets n'ont pas de méthode main(), ils sont contrôlés par le conteneur Tomcat
- Les requêtes ne sont pas adressées aux servlets mais au conteneur dans lequel ils sont déployés

Conteneur

- Un conteneur fournit pour les Servlets Un support pour la communication :
- Pas besoin de ServerSocket, Socket, Stream, ...
- La gestion du cycle de vie
- Un support pour le Multithreading (Création automatique des Threads)
- Un support pour la sécurité
- Un support pour les JSP

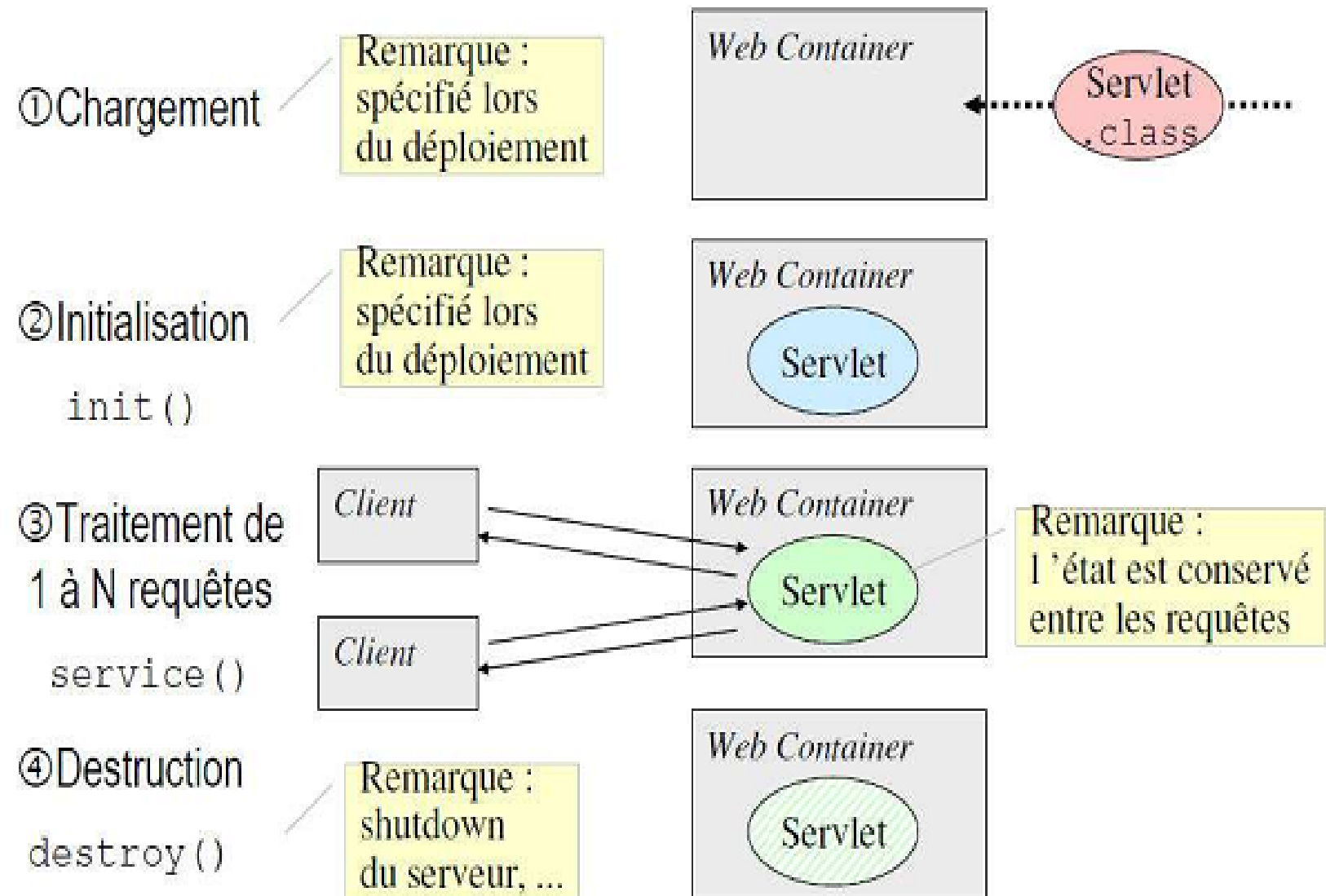
- Un servlet est un objet qui peut être manipulé par le conteneur via l'interface suivante :



- Lorsque le conteneur reçoit une requête, il la transmet au servlet qui correspond à l'URL pour que la requête soit traitée effectivement

Cycle de Vie d'un servlet

- Cycle de Vie d'un servlet
 - 1 Chargement de la classe
 - 2 Instanciation du servlet (constructeur par défaut)
 - 3 Appel de init()
 - 4 Appel(s) de service() (1 thread par requête)
 - 5 Appel de destroy()

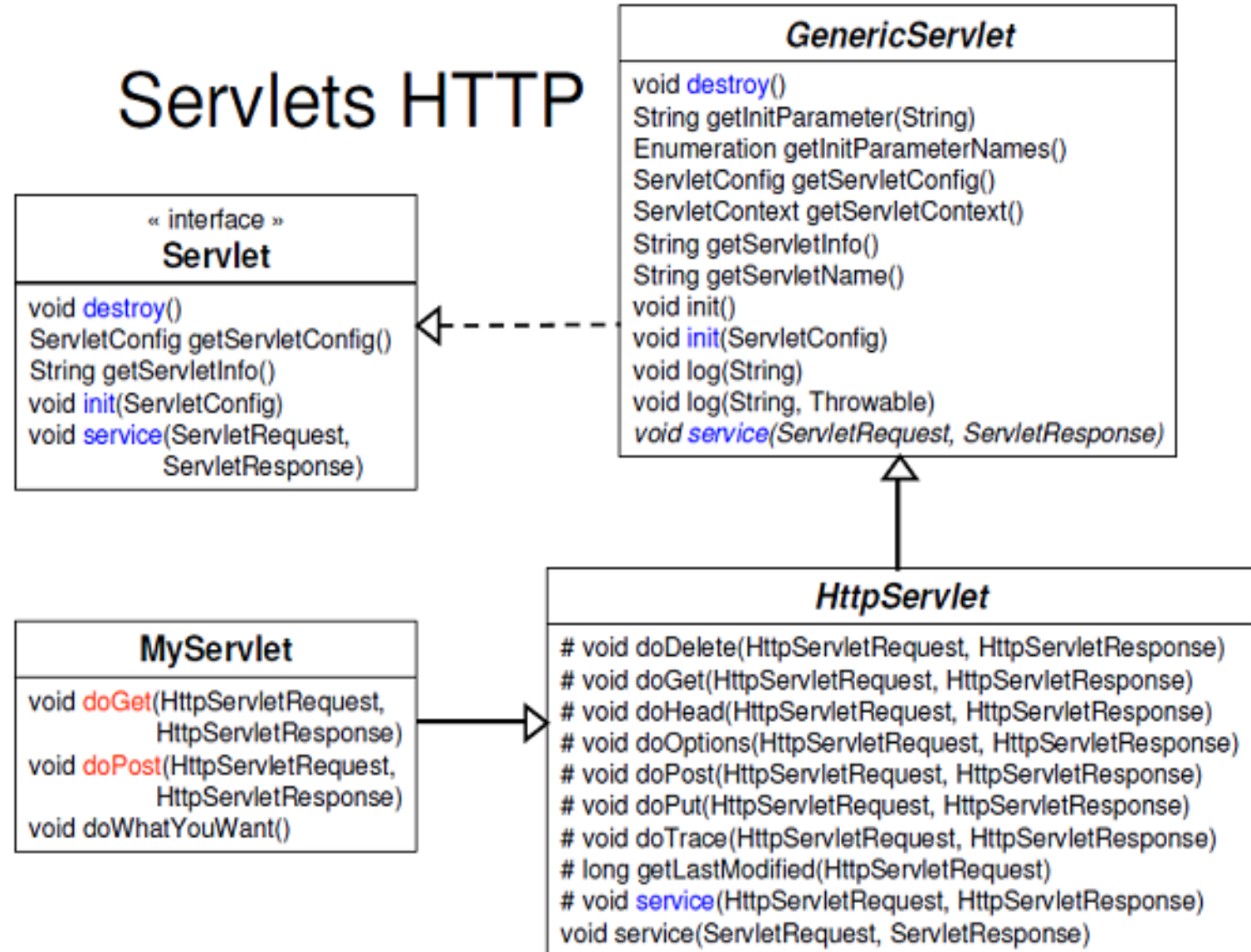


La méthode service()

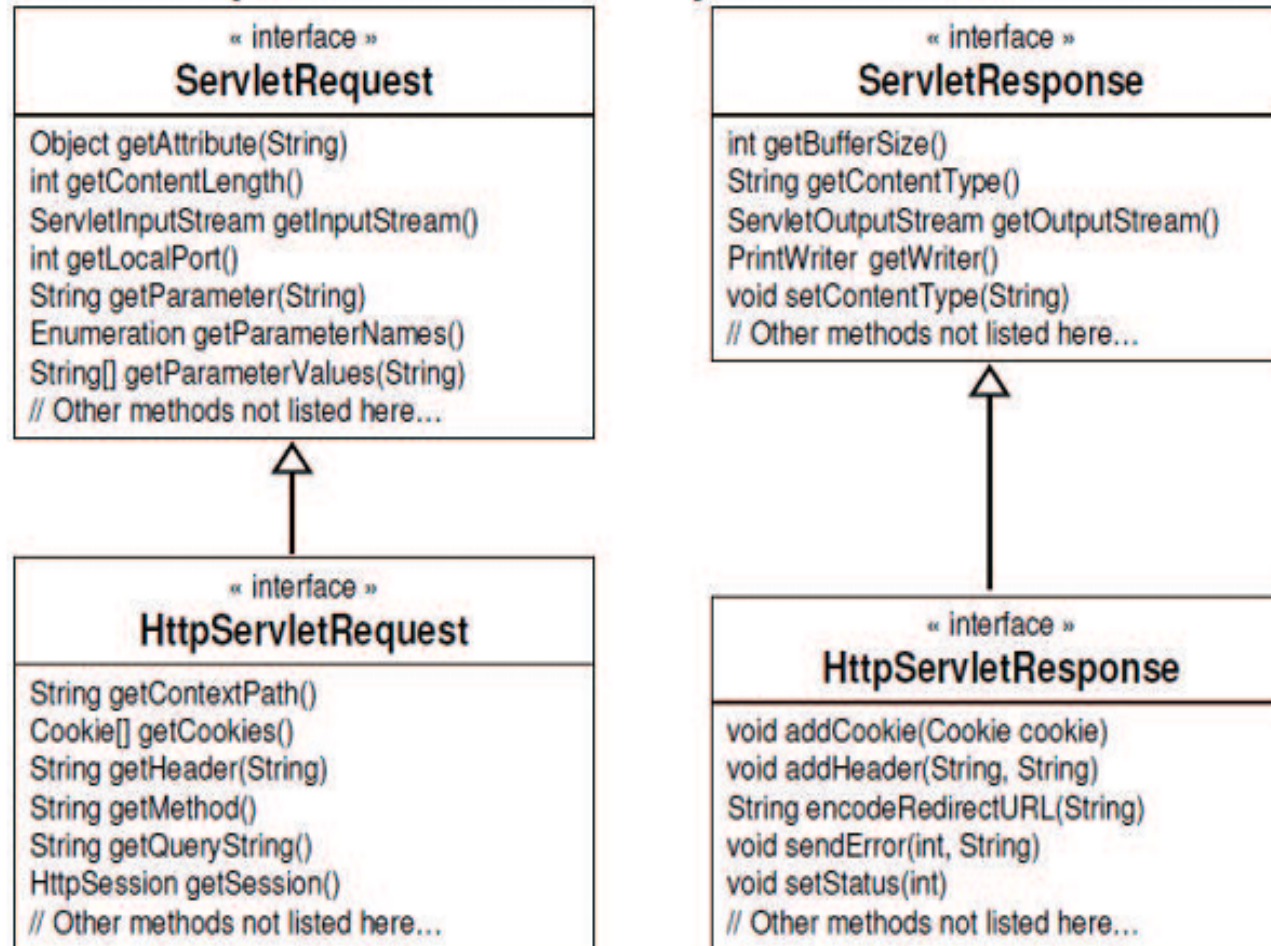
- Lors de la réception d'une requête, le conteneur crée
 - Un objet `ServletRequest` (la requête), et
 - Un objet `ServletResponse` (la réponse)
- Le conteneur appelle ensuite la méthode `service()` avec ces deux objets en paramètres pour permettre au servlet de répondre à la requête du client

L'API Servlet

Servlets HTTP

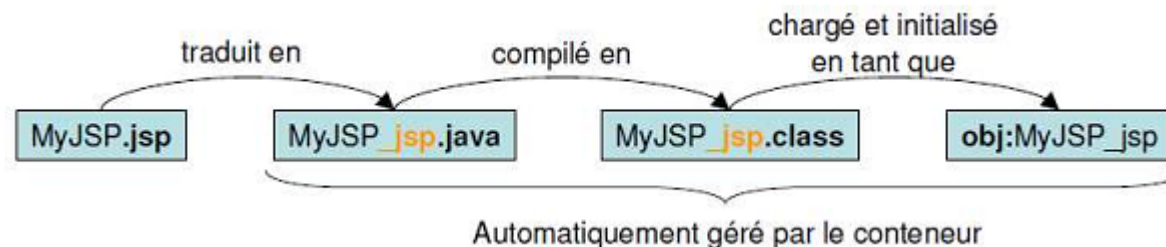


L'API Servlet



JSP

- Ecrire une servlet $\Rightarrow$ écrire une classe et la compiler
- `out. println (" < html >< head >< title >" + title + " ... ");`
- Les JSP permettent d'intégrer du code java dans une page HTML entre les balises .
- **`<h1 > Time on server </h1 >`**
- **`<p > <%= new java . util . Date () % > </p>`**
- **C'est le serveur d'application qui va se charger de la**
- **transformer en Servlet et de la compiler quand il le faut**
- **Mais au final une JSP n'est qu'un servlet !**



Mécanismes mis en oeuvre

- lors du premier chargement d'une JSP (ou après modification), le moteur :
 - rassemble tous les fragments de la JSP dans une classe
 - la compile
 - l'instancie
 - $\Rightarrow$ JSP = objet Java présent dans le moteur
- puis, ou lors des chargements suivants, le moteur :
 - exécute le code dans un thread
 - délai d'attente lors de la 1ère invocation dû à la compilation en cas d'erreur de syntaxe dans le code Java de la JSP message récupéré dans le navigateur

Correspondance JSP/Servlet

- JSP d'origine **Un nombre aléatoire**

- Code du servlet résultant de la traduction

```
public void _jspService ( HttpServletRequest request , HttpServletResponse  
response )
```

```
throws ServletException , IOException { response . setContentType (" text / html  
"); HttpSession session = request . getSession (); JspWriter out = response .  
getWriter ();
```

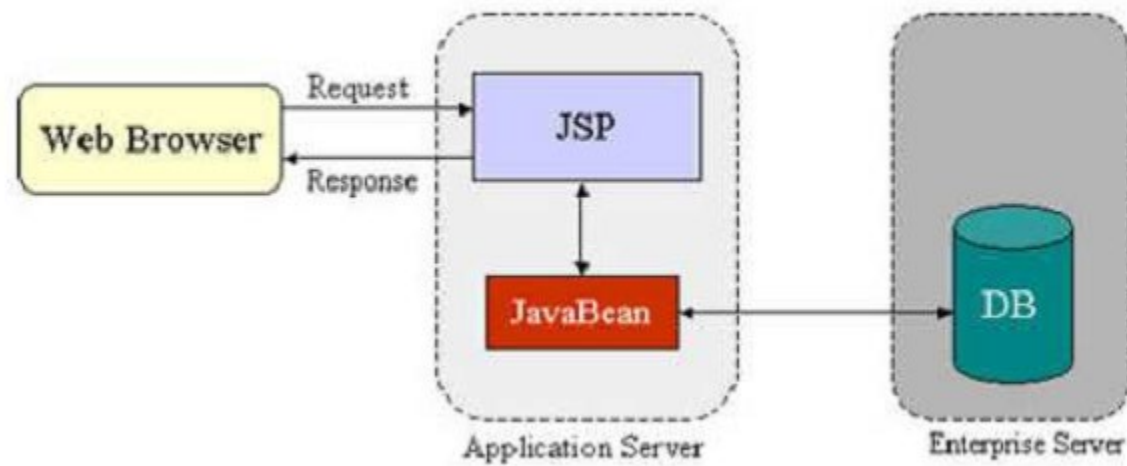
```
out. println (" Un _ nombre _ aléatoire ");
```

```
out. println ( Math . random ()); ... }
```

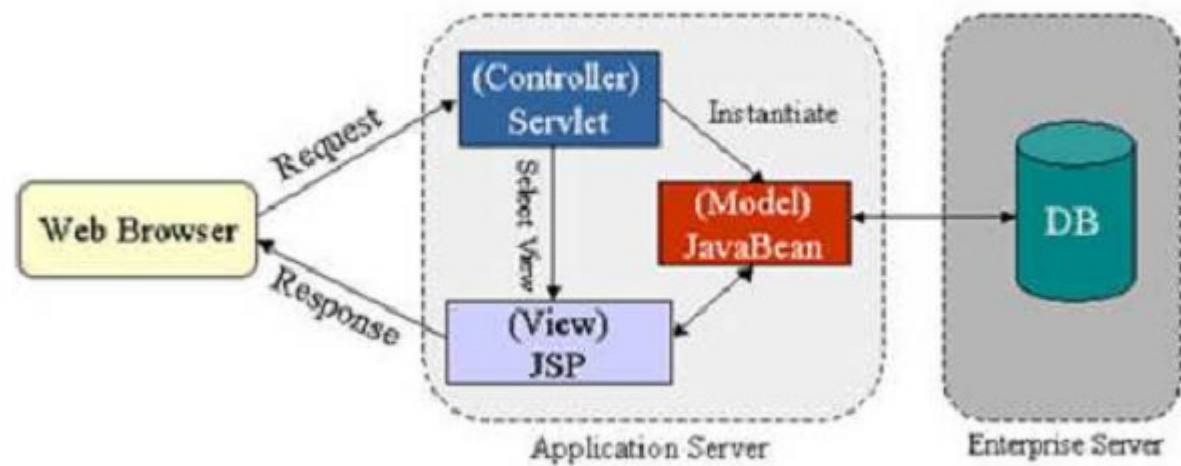
Java Server Pages Model Architecture

- Les premières spécifications JSP préconise deux approches de création d'applications utilisant la technologie JSP [1] :
- JSP Model 1 architecture :
 - la page JSP est seul responsable du traitement de la requête entrante et de la réponse au client.
 - Adapté pour des applications simples
- JSP Model 2 architecture :
 - Combine l'utilisation des servlets et JSP
 - JSP pour générer la couche de présentation Servlets agit en tant que contrôleur et se charge de :
 - Traitement des requêtes
 - Création des Beans ou des objets utilisés par les JSP,
 - Décider, en fonction des actions de l'utilisateur, quelle page JSP transmettre

Architecture Web JEE : Model 1 (sans MVC)



Architecture Web JEE : Model 2 (MVC)



Pourquoi combiner Servlets & JSP ?

- Servlet seuls. Marche bien quand :
 - L'output est de type binaire. Ex : une image
 - Il n'y a pas d'output. Ex : redirections
 - La forme/présentation est variable. Ex : portail
- JSP seules. Marche bien quand :
 - L'output est de type caractère. Ex : HTML
 - La forme/présentation est stable.

Pourquoi combiner Servlets & JSP ? (suite)

- Architecture MVC. Nécessaire quand :
 - Une même requête peut donner des résultats visuels réellement différents
 - Equipe de développement avec une partie pour le développement Web et une autre pour la logique métier
 - Un traitement complexe des données mais une présentation relativement fixe

Notion de module Web (.war)

- Un servlet ne peut pas être déployé directement dans un conteneur, il doit faire partie d'un module Web
- Un module Web est un ensemble de librairies, de fichiers de configurations, de code Java (bytecode des servlets), JavaServer Pages, servlets, classes Java, fichiers XML, et des pages web statiques (HTML, Javascript, ...)
- le tout constituant une application web.

Notion de module Web (.war) (suite)

- Le module Web (fichier WAR, pour Web Application ARchive, est utilisé pour déployer une application web sur un serveur d'application (Tomcat ou GlassFish). On y trouve toujours

WEB-INF/

- web.xml

- lib/

- Classes/

Notion de module Web (.war)

- web.xml
 - Est un fichier XML de description de l'application
 - Il a une syntaxe précise
 - On va, par ex., y trouver le nom de la Classe (implémentant l'interface Servlet) à utiliser pour tel ou tel chemin d'une URL Web

- Lib/
 - On va y mettre tous les .jar dont notre programme a besoin
- Classes/
 - On va y mettre toutes les .class que l'on produit
- Le serveur d'applications va savoir, pour chaque application qu'il héberge, lire le web.xml et créer automatiquement un classpath spécifique à l'application à partir des répertoires lib et classes

- [http ://www.javaworld.com/javaworld/jw-12-1999/jw-12-ssjjspmvc.htm](http://www.javaworld.com/javaworld/jw-12-1999/jw-12-ssjjspmvc.htm)
- Des transparents et des figures empruntés à L. Seinturier, M. Buffa et F. Romeo